Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Department of Artificial Intelligence (AI) & Data Science

Name: Aaryan Nandkishor Raorane	**Roll no: S006**
Sap ID: 60018250070	**Topic: Data Structure (Doubly Linked List)**
Branch: Artificial Intelligence & Data Science	

Experiment No. 6

Aim: To implement a Doubly Linked List (DLL) that simulates an image viewer application, supporting forward and backward navigation between images along with insertion, deletion, sorting, combining, and additional utility operations.

Theory:

Definition:

A doubly linked list is a linear dynamic data structure in which each node contains a data field and two pointers: next pointing to the successor node, and prev pointing to the predecessor node. The head node's prev holds NULL and the tail node's next holds NULL. Because each node is independently allocated on the heap, the list grows or shrinks at runtime without contiguous storage. Bidirectional traversal (both forward and backward) is the key advantage over a singly linked list. Here, each node stores an image name and two pointers enabling navigation in both directions.

Node Structure:-

C Struct:

```
struct Node {
    char imgName[100];
    struct Node *prev;
    struct Node *next;
};
```

Operations:

InsertImage

Images may be inserted at the end, at the beginning, or at any specified position.

1. Insert at End

Pseudocode:

```
BEGIN InsertAtEnd(HEAD, IMG_NAME)
    NEW_NODE = ALLOCATE NODE()
    NEW_NODE.imgName = IMG_NAME
    NEW_NODE.next = NULL
    NEW_NODE.prev = NULL
    IF HEAD == NULL THEN
        HEAD = NEW_NODE
    ELSE
        SET TEMP = HEAD
        WHILE TEMP.next != NULL DO
            TEMP = TEMP.next
        END WHILE
        TEMP.next = NEW_NODE
        NEW_NODE.prev = TEMP
    END IF
    PRINT "Image added at end"
END InsertAtEnd
```

2. Insert at Beginning

Pseudocode:

```
BEGIN InsertAtBeginning(HEAD, IMG_NAME)
    NEW_NODE = ALLOCATE NODE()
    NEW_NODE.imgName = IMG_NAME
    NEW_NODE.prev = NULL
    NEW_NODE.next = HEAD
    IF HEAD != NULL THEN
        HEAD.prev = NEW_NODE
```

```
    END IF
    HEAD = NEW_NODE
    PRINT "Image added at beginning"
  END InsertAtBeginning
```

3. Insert at Position

Pseudocode:

```
  BEGIN InsertAtPosition(HEAD, IMG_NAME, POS)
    NEW_NODE = ALLOCATE NODE()
    NEW_NODE.imgName = IMG_NAME
    IF POS == 1 THEN
      NEW_NODE.next = HEAD
      NEW_NODE.prev = NULL
      IF HEAD != NULL THEN
        HEAD.prev = NEW_NODE
      END IF
      HEAD = NEW_NODE
    ELSE
      SET TEMP = HEAD
      SET I = 1
      WHILE I < POS - 1 AND TEMP != NULL DO
        TEMP = TEMP.next
        I = I + 1
      END WHILE
      IF TEMP == NULL THEN
        PRINT "Position out of range"
        FREE NEW_NODE
      ELSE
        NEW_NODE.next = TEMP.next
        NEW_NODE.prev = TEMP
        IF TEMP.next != NULL THEN
          TEMP.next.prev = NEW_NODE
        END IF
```

```
        TEMP.next = NEW_NODE
      END IF
    END IF
  END InsertAtPosition
```

DeleteImage

Images may be removed from the beginning, the end, or by matching the image name.

1. Delete at Beginning

Pseudocode:

```
  BEGIN DeleteAtBeginning(HEAD)
    IF HEAD == NULL THEN
      PRINT "Viewer is empty"
    ELSE
      TEMP = HEAD
      HEAD = HEAD.next
      IF HEAD != NULL THEN
        HEAD.prev = NULL
      END IF
      PRINT "Deleted: " + TEMP.imgName
      FREE TEMP
    END IF
  END DeleteAtBeginning
```

2. Delete at End

Pseudocode:

```
  BEGIN DeleteAtEnd(HEAD)
    IF HEAD == NULL THEN
      PRINT "Viewer is empty"
    ELSE IF HEAD.next == NULL THEN
      PRINT "Deleted: " + HEAD.imgName
      FREE HEAD
```

```
    HEAD = NULL
  ELSE
    SET TEMP = HEAD
    WHILE TEMP.next != NULL DO
      TEMP = TEMP.next
    END WHILE
    TEMP.prev.next = NULL
    PRINT "Deleted: " + TEMP.imgName
    FREE TEMP
  END IF
END DeleteAtEnd
```

3. Delete by Image Name

Pseudocode:

```
BEGIN DeleteByName(HEAD, IMG_NAME)
  SET TEMP = HEAD
  WHILE TEMP != NULL DO
    IF TEMP.imgName == IMG_NAME THEN
      IF TEMP.prev != NULL THEN
        TEMP.prev.next = TEMP.next
      ELSE
        HEAD = TEMP.next
      END IF
      IF TEMP.next != NULL THEN
        TEMP.next.prev = TEMP.prev
      END IF
      PRINT "Deleted: " + TEMP.imgName
      FREE TEMP
      RETURN
    END IF
    TEMP = TEMP.next
  END WHILE
  PRINT "Image not found"
```

```
END DeleteByName
```

NavigateImage

The viewer supports one-step navigation to the next or previous image from any given image.

1. Navigate Forward (Next)

Pseudocode:

```
BEGIN NavigateForward(HEAD, IMG_NAME)
  SET TEMP = HEAD
  WHILE TEMP != NULL DO
    IF TEMP.imgName == IMG_NAME THEN
      IF TEMP.next != NULL THEN
        PRINT "Next: " + TEMP.next.imgName
      ELSE
        PRINT "Already at last image"
      END IF
      RETURN
    END IF
    TEMP = TEMP.next
  END WHILE
  PRINT "Image not found"
END NavigateForward
```

2. Navigate Backward (Previous)

Pseudocode:

```
BEGIN NavigateBackward(HEAD, IMG_NAME)
  SET TEMP = HEAD
  WHILE TEMP != NULL DO
    IF TEMP.imgName == IMG_NAME THEN
      IF TEMP.prev != NULL THEN
        PRINT "Previous: " + TEMP.prev.imgName
      ELSE
```

```
            PRINT "Already at first image"
          END IF
          RETURN
        END IF
        TEMP = TEMP.next
      END WHILE
      PRINT "Image not found"
    END NavigateBackward
```

DisplayList

DisplayForward traverses from head to tail; DisplayBackward traverses from tail to head using the prev pointer.

1. Display Forward

Pseudocode:

```
    BEGIN DisplayForward(HEAD)
      IF HEAD == NULL THEN
        PRINT "Viewer is empty"
      ELSE
        SET TEMP = HEAD
        WHILE TEMP != NULL DO
          PRINT TEMP.imgName + " <-> "
          TEMP = TEMP.next
        END WHILE
        PRINT "NULL"
      END IF
    END DisplayForward
```

2. Display Backward

Pseudocode:

```
    BEGIN DisplayBackward(HEAD)
      IF HEAD == NULL THEN
        PRINT "Viewer is empty"
```

```
    ELSE
      SET TEMP = HEAD
      WHILE TEMP.next != NULL DO
        TEMP = TEMP.next
      END WHILE
      WHILE TEMP != NULL DO
        PRINT TEMP.imgName + " <-> "
        TEMP = TEMP.prev
      END WHILE
      PRINT "NULL"
    END IF
  END DisplayBackward
```

Additional Operations

CountImages returns the total node count; CheckPalindrome checks whether the list reads the same forward and backward; SwapNodes exchanges two nodes by relinking pointers without touching data; CombineImages merges two specified image nodes into one; SortImagesByName arranges all nodes in alphabetical order by image name.

1. CountImages

Pseudocode:

```
  BEGIN CountImages(HEAD)
    SET TEMP = HEAD
    SET COUNT = 0
    WHILE TEMP != NULL DO
      COUNT = COUNT + 1
      TEMP = TEMP.next
    END WHILE
    PRINT "Total images: " + COUNT
    RETURN COUNT
  END CountImages
```

2. CheckPalindrome

Pseudocode:

```
BEGIN CheckPalindrome(HEAD)
  IF HEAD == NULL THEN
    PRINT "List is empty"
    RETURN
  END IF
  SET FRONT = HEAD
  SET BACK = HEAD
  WHILE BACK.next != NULL DO
    BACK = BACK.next
  END WHILE
  WHILE FRONT != BACK AND FRONT.prev != BACK DO
    IF FRONT.imgName != BACK.imgName THEN
      PRINT "Not a Palindrome"
      RETURN
    END IF
    FRONT = FRONT.next
    BACK = BACK.prev
  END WHILE
  PRINT "Palindrome!"
END CheckPalindrome
```

3. SwapNodes

Pseudocode:

```
BEGIN SwapNodes(HEAD, NAME1, NAME2)
  SET A = HEAD
  SET B = HEAD
  WHILE A != NULL AND A.imgName != NAME1 DO
    A = A.next
  END WHILE
  WHILE B != NULL AND B.imgName != NAME2 DO
    B = B.next
```

```
    END WHILE
    IF A == NULL OR B == NULL THEN
        PRINT "Node not found"
        RETURN
    END IF
    IF A == B THEN RETURN END IF
    IF A.prev != NULL THEN A.prev.next = B ELSE HEAD = B END IF
    IF B.prev != NULL THEN B.prev.next = A ELSE HEAD = A END IF
    SWAP A.prev AND B.prev
    SWAP A.next AND B.next
    IF A.next != NULL THEN A.next.prev = A END IF
    IF B.next != NULL THEN B.next.prev = B END IF
    PRINT "Nodes swapped"
END SwapNodes
```

4. CombineImages

Merges two specified image nodes into a single node whose name is the concatenation of both names (e.g., "Sunset.jpg" + "Ocean.jpg" becomes "Sunset+Ocean.jpg"). The second node is removed from the list.

Pseudocode:

```
BEGIN CombineImages(HEAD, NAME1, NAME2)
    SET A = HEAD
    SET B = HEAD
    WHILE A != NULL AND A.imgName != NAME1 DO
        A = A.next
    END WHILE
    WHILE B != NULL AND B.imgName != NAME2 DO
        B = B.next
    END WHILE
    IF A == NULL OR B == NULL OR A == B THEN
        PRINT "Invalid images"
        RETURN
    END IF
```

```
    NEW_NAME = A.imgName + "+" + B.imgName
    IF A.next == B THEN
        A.next = B.next
        IF B.next != NULL THEN B.next.prev = A END IF
    ELSE IF B.next == A THEN
        B.next = A.next
        IF A.next != NULL THEN A.next.prev = B END IF
        A = B
    ELSE
        IF A.prev != NULL THEN A.prev.next = A.next ELSE HEAD =
A.next END IF
        IF A.next != NULL THEN A.next.prev = A.prev END IF
        IF B.prev != NULL THEN B.prev.next = B.next ELSE HEAD =
B.next END IF
        IF B.next != NULL THEN B.next.prev = B.prev END IF
        A = B
    END IF
    A.imgName = NEW_NAME
    PRINT "Combined into: " + NEW_NAME
END CombineImages
```

5. SortImagesByName

Sorts all nodes in ascending alphabetical order by image name using bubble sort on node data (swapping imgName strings, not relinking pointers).

Pseudocode:

```
BEGIN SortImagesByName(HEAD)
    IF HEAD == NULL THEN
        PRINT "List is empty"
        RETURN
    END IF
    SET SWAPPED = TRUE
    WHILE SWAPPED == TRUE DO
        SWAPPED = FALSE
```

```
        SET TEMP = HEAD
        WHILE TEMP.next != NULL DO
          IF TEMP.imgName > TEMP.next.imgName THEN
            SWAP TEMP.imgName AND TEMP.next.imgName
            SWAPPED = TRUE
          END IF
          TEMP = TEMP.next
        END WHILE
      END WHILE
      PRINT "Images sorted by name"
    END SortImagesByName
```

Code:

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>
#include<string.h>

struct Node {
   char imgName[100];
   struct Node *prev;
   struct Node *next;
};

struct Node *head = NULL;

void insertAtEnd() {
   struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
   printf("Enter image name: "); gets(newNode->imgName);
   newNode->next = NULL;
   newNode->prev = NULL;
   if(head == NULL) { head = newNode; return; }
```

```
    struct Node *temp = head;
    while(temp->next != NULL) temp = temp->next;
    temp->next = newNode;
    newNode->prev = temp;
    printf("Image added.\n");
}

void insertAtBeginning() {
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    printf("Enter image name: "); gets(newNode->imgName);
    newNode->prev = NULL;
    newNode->next = head;
    if(head != NULL) head->prev = newNode;
    head = newNode;
    printf("Image added at beginning.\n");
}

void insertAtPosition() {
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    int pos, i;
    printf("Enter image name: "); gets(newNode->imgName);
    printf("Enter position: "); scanf("%d", &pos); getchar();
    if(pos == 1) { newNode->next = head; newNode->prev = NULL;
        if(head != NULL) head->prev = newNode; head = newNode; return;
}
    struct Node *temp = head;
    for(i = 1; i < pos - 1 && temp != NULL; i++) temp = temp->next;
    if(temp == NULL) { printf("Position out of range.\n"); free(newNode);
return; }
    newNode->next = temp->next;
    newNode->prev = temp;
    if(temp->next != NULL) temp->next->prev = newNode;
    temp->next = newNode;
```

```
    printf("Image added at position %d.\n", pos);
}

void deleteAtBeginning() {
    if(head == NULL) { printf("Viewer is empty.\n"); return; }
    struct Node *temp = head;
    head = head->next;
    if(head != NULL) head->prev = NULL;
    printf("Deleted: %s\n", temp->imgName);
    free(temp);
}

void deleteAtEnd() {
    if(head == NULL) { printf("Viewer is empty.\n"); return; }
    if(head->next == NULL) { printf("Deleted: %s\n", head->imgName);
free(head); head = NULL; return; }
    struct Node *temp = head;
    while(temp->next != NULL) temp = temp->next;
    temp->prev->next = NULL;
    printf("Deleted: %s\n", temp->imgName);
    free(temp);
}

void deleteByName() {
    char name[100];
    printf("Enter image name to delete: "); gets(name);
    struct Node *temp = head;
    while(temp != NULL) {
        if(strcmp(temp->imgName, name) == 0) {
            if(temp->prev != NULL) temp->prev->next = temp->next;
            else head = temp->next;
            if(temp->next != NULL) temp->next->prev = temp->prev;
            printf("Deleted: %s\n", temp->imgName);
```

```
            free(temp); return;
        }
        temp = temp->next;
    }
    printf("Image not found.\n");
}

void navigateForward() {
    char name[100];
    printf("Enter current image: "); gets(name);
    struct Node *temp = head;
    while(temp != NULL) {
        if(strcmp(temp->imgName, name) == 0) {
            if(temp->next != NULL) printf("Next image: %s\n", temp->next->imgName);
            else printf("Already at last image.\n");
            return;
        }
        temp = temp->next;
    }
    printf("Image not found.\n");
}

void navigateBackward() {
    char name[100];
    printf("Enter current image: "); gets(name);
    struct Node *temp = head;
    while(temp != NULL) {
        if(strcmp(temp->imgName, name) == 0) {
            if(temp->prev != NULL) printf("Previous image: %s\n", temp->prev->imgName);
            else printf("Already at first image.\n");
            return;
```

```
        }
        temp = temp->next;
    }
    printf("Image not found.\n");
}

void displayForward() {
    if(head == NULL) { printf("Viewer is empty.\n"); return; }
    struct Node *temp = head;
    printf("Forward: ");
    while(temp != NULL) { printf("%s <-> ", temp->imgName); temp =
temp->next; }
    printf("NULL\n");
}

void displayBackward() {
    if(head == NULL) { printf("Viewer is empty.\n"); return; }
    struct Node *temp = head;
    while(temp->next != NULL) temp = temp->next;
    printf("Backward: ");
    while(temp != NULL) { printf("%s <-> ", temp->imgName); temp =
temp->prev; }
    printf("NULL\n");
}

int countImages() {
    struct Node *temp = head; int count = 0;
    while(temp != NULL) { count++; temp = temp->next; }
    return count;
}

void checkPalindrome() {
    struct Node *front = head, *back = head;
```

```
    if(head == NULL) { printf("List is empty.\n"); return; }
    while(back->next != NULL) back = back->next;
    while(front != back && front->prev != back) {
        if(strcmp(front->imgName, back->imgName) != 0) {
            printf("Not a Palindrome.\n"); return;
        }
        front = front->next;
        back = back->prev;
    }
    printf("Palindrome!\n");
}

void swapNodes() {
    char n1[100], n2[100];
    printf("Enter first image: "); gets(n1);
    printf("Enter second image: "); gets(n2);
    struct Node *a = head, *b = head;
    while(a != NULL && strcmp(a->imgName, n1) != 0) a = a->next;
    while(b != NULL && strcmp(b->imgName, n2) != 0) b = b->next;
    if(a == NULL || b == NULL) { printf("Node not found.\n"); return; }
    if(a == b) return;
    if(a->prev != NULL) a->prev->next = b;
    else head = b;
    if(b->prev != NULL) b->prev->next = a;
    else head = a;
    struct Node *tmp;
    tmp = a->prev; a->prev = b->prev; b->prev = tmp;
    tmp = a->next; a->next = b->next; b->next = tmp;
    if(a->next != NULL) a->next->prev = a;
    if(b->next != NULL) b->next->prev = b;
    printf("Nodes swapped.\n");
}
```

```
void combineImages() {
    char n1[100], n2[100], newName[200];
    printf("Enter first image name: "); gets(n1);
    printf("Enter second image name: "); gets(n2);
    struct Node *a = head, *b = head;
    while(a != NULL && strcmp(a->imgName, n1) != 0) a = a->next;
    while(b != NULL && strcmp(b->imgName, n2) != 0) b = b->next;
    if(a == NULL || b == NULL || a == b) { printf("Invalid images.\n");
return; }
    strcpy(newName, a->imgName);
    strcat(newName, "+");
    strcat(newName, b->imgName);
    if(a->next == b) {
        a->next = b->next;
        if(b->next != NULL) b->next->prev = a;
    } else if(b->next == a) {
        b->next = a->next;
        if(a->next != NULL) a->next->prev = b;
        a = b;
    } else {
        if(a->prev != NULL) a->prev->next = a->next; else head = a->next;
        if(a->next != NULL) a->next->prev = a->prev;
        if(b->prev != NULL) b->prev->next = b->next; else head = b->next;
        if(b->next != NULL) b->next->prev = b->prev;
        a = b;
    }
    strcpy(a->imgName, newName);
    printf("Combined into: %s\n", newName);
}

void sortImagesByName() {
    int swapped;
    struct Node *temp;
```

```
    char tempName[100];
    if(head == NULL) { printf("List is empty.\n"); return; }
    do {
        swapped = 0;
        temp = head;
        while(temp->next != NULL) {
            if(strcmp(temp->imgName, temp->next->imgName) > 0) {
                strcpy(tempName, temp->imgName);
                strcpy(temp->imgName, temp->next->imgName);
                strcpy(temp->next->imgName, tempName);
                swapped = 1;
            }
            temp = temp->next;
        }
    } while(swapped);
    printf("Images sorted by name.\n");
}

void main() {
    int choice; clrscr();
    do {
        printf("\n--- Image Viewer ---\n");
        printf("1.Add End  2.Add Begin  3.Add Pos\n");
        printf("4.Del Begin  5.Del End  6.Del Name\n");
        printf("7.Next  8.Prev  9.Display Fwd  10.Display Bwd\n");
        printf("11.Count  12.Palindrome  13.Swap\n");
        printf("14.Combine  15.Sort  0.Exit\n");
        printf("Choice: "); scanf("%d", &choice); getchar();
        switch(choice) {
            case 1: insertAtEnd(); break;
            case 2: insertAtBeginning(); break;
            case 3: insertAtPosition(); break;
            case 4: deleteAtBeginning(); break;
```

```
            case 5: deleteAtEnd(); break;
            case 6: deleteByName(); break;
            case 7: navigateForward(); break;
            case 8: navigateBackward(); break;
            case 9: displayForward(); break;
            case 10: displayBackward(); break;
            case 11: printf("Total: %d\n", countImages()); break;
            case 12: checkPalindrome(); break;
            case 13: swapNodes(); break;
            case 14: combineImages(); break;
            case 15: sortImagesByName(); break;
            case 0: break;
            default: printf("Invalid.\n");
        }
    } while(choice != 0);
    getch();
}
```

Output:

```
--- Image Viewer ---
1.Add End  2.Add Begin  3.Add Pos
4.Del Begin  5.Del End  6.Del Name
7.Next  8.Prev  9.Display Fwd  10.Display Bwd
11.Count  12.Palindrome  13.Swap
14.Combine  15.Sort  0.Exit
Choice: 1
Enter image name: Sunset.jpg
Image added.

Choice: 1
Enter image name: Mountains.jpg
Image added.
```

```
Choice: 1
Enter image name: Ocean.jpg
Image added.

Choice: 9
Forward: Sunset.jpg <-> Mountains.jpg <-> Ocean.jpg <-> NULL

Choice: 15
Images sorted by name.

Choice: 9
Forward: Mountains.jpg <-> Ocean.jpg <-> Sunset.jpg <-> NULL

Choice: 14
Enter first image name: Mountains.jpg
Enter second image name: Ocean.jpg
Combined into: Mountains.jpg+Ocean.jpg

Choice: 9
Forward: Mountains.jpg+Ocean.jpg <-> Sunset.jpg <-> NULL

Choice: 0
```

Applications:

Doubly linked lists are used in image and media viewers (forward/backward navigation), browser history (back/forward buttons), music players with previous/next track controls, OS process scheduling (allowing both insertion and deletion from either end), and text editor undo/redo chains. The new CombineImages operation supports merging related images (e.g., panoramas or before/after pairs), while SortImagesByName enables alphabetical organization of galleries.

Conclusion:

This experiment implemented a doubly linked list to simulate an image viewer. Fifteen operations were coded in C: three insertions, three deletions, forward/backward navigation, two display traversals, count, palindrome check, pointer-only node swap, CombineImages (merging two nodes into one), and SortImagesByName (bubble-sort on node data). The prev pointer enables O(1) predecessor access and bidirectional traversal, while the new operations demonstrate practical image-gallery management features.